

Shihan Sharar

☎ (548) 577-1483 | ✉ ssharar@uwaterloo.ca | 🏠 s-sharar.vercel.app | 🌐 s-sharar | in shihan-sharar

Education

University of Waterloo

Waterloo, ON

Bachelor of Computer Science

Sept 2023 - Apr 2028 (Expected)

- **Dean's List** (all terms); CGPA: **94%**, Faculty GPA: **96%** (with advanced courses)
- **Clubs:** Measuring rover inclination angle for UW Robotics Club, member of Waterloo Quant and UW Putnam Club
- **Skills:** C++, C, C#, Python, Java, Go, Ruby, Rust, React, TypeScript/JavaScript, Node.js

Experience

Persona

San Francisco, California

Software Engineer Intern - Platform Experience

Jun 2026 - Sep 2026

- Re-architected event retrieval in **Ruby on Rails and TypeScript**, delivering **2.4×** faster production API responses with permission-scoped cursor pagination across **MongoDB and Snowflake**.
- Added **polling** for live timeline updates and **batched, cached** actor lookups to avoid per-actor requests.
- Built runtime infrastructure for Layout Editor computed properties, executing **JSON**-logic and sandboxed **QuickJS** custom code in CSV export pipelines to enforce dynamic visibility rules outside the browser.
- Developed an **AI eval** harness on **Cloudflare** to compare models/skill versions, running **Cursor SDK** evals in parallel isolated containers with build checks, **LLM-as-judge** scoring, and **TSX AST scorers** for design-system usage.

University of Waterloo

Waterloo, ON

Compiler Research Assistant - Prof. Y. Zhang

Jan 2025 - Apr 2026

- Built a **Python** compiler pass for a probabilistic programming language (**PPL**) that lowers continuous distributions into fixed-point Boolean arithmetic for **JAX/XLA** inference.
- Implemented bit-blasting for **exponential**, **Laplace**, and **gamma** distributions, and extended the compiler to arbitrary densities using **CDF**-fitted exponential segments with interleaved variable elimination to bound tensor size.

BitGo

Palo Alto, California

Software Engineer Intern - Prime Trade

Sep 2025 - Dec 2025

- Designed and engineered an **LP-settlement** tracking system in **Go**, introducing new data models in **PostgreSQL**, **GraphQL** schemas/resolvers, and APIs to track settlement lifecycle state and reconcile trades.
- Developed an ingestion framework for **Talos** OTC fills, producing normalized data for the settlement system.
- Built **React** dashboards with filtering and sorting over ingestion and settlement data, cutting operational settlement workflows by **~20 minutes** per case; validated with **Vitest**.
- Implemented a notification pipeline, defining **gRPC** schemas, producers that write to an **outbox**, a publisher that streams events to **Kafka**, and consumers that trigger **Novu** workflows for over **10,000** notifications daily.
- Eliminated duplicate notifications and hardened delivery across failures using deterministic **idempotency** keys, persisted delivery state, and exponential-backoff retries with a DLQ.

University of Waterloo

Waterloo, ON

System Security Research Assistant - Prof. N. Asokan

May 2025 - Aug 2025

- Engineered a **C++** command-line interface using the **llama.cpp** API, adding an executor framework that maps model outputs to deterministic system actions with full control-flow tracing.
- Built a secure-world **OP-TEE** monitor in **C** (emulated ARM TrustZone) to validate all LLM interactions via signing/ hashing, preventing indirect prompt-injection and enforcing control-flow integrity (**CFI**).

Projects

Volguard: BTC Options IV Surface Forecasting



- Engineered a **BTC** options data platform that backfilled **Deribit** market data, streamed **20M+** option chain rows into partitioned **Parquet**, and produced **2,019** leakage-audited daily implied-volatility surfaces.
- Implemented **Black-76** pricing/Greeks, **SVI** calibration, butterfly/calendar checks, and **OSQP** repair with a **walk-forward evaluation** framework to benchmark five baselines against residual **GRU** and constrained-decoder models.

Velocore: Multithreaded C++ Trading Engine



- Built a multithreaded trading engine using **Crow**, **Boost.Asio**, & **Boost.Beast** for async I/O, & websocket communication; processing **1,000+** orders in under **30ms** with **per-symbol locking** for concurrent execution.
- Built a custom broker interface over **Alpaca APIs** for live market data and paper order execution.